Visitor Pattern

The Visitor pattern is a behavioral design pattern that allows you to add new actions to existing object structures without altering them. It consists of two important components: the items being operated on and the visitors who conduct operations on these elements.

Consider an online shopping platform that offers different discount schemes based on the kind of things bought (e.g., groceries, electronics, or apparel) and the customer status (e.g., new customer, loyal customer, or VIP customer).

Visitor Pattern - Types of Customers

Implementing and maintaining these discount strategies directly inside the customer and product classes might result in a complicated and tightly coupled system, especially when new discount strategies or customer groups are routinely added.

The Visitor pattern is used to externalize discount strategies from the customer and product classes. This is achieved by:

- **Creating Visitors for Discounts**: Separate classes (visitors) are created for each discount strategy. These visitors encapsulate the logic to calculate discounts based on different criteria.
- **Visitable Elements (Customers and Products)**: Customer and product classes are designed to accept visitors. These classes have a method that allows a visitor to perform operations on them.
- **Applying Discounts**: When a discount needs to be applied, the appropriate visitor is passed to the customer or product. The visitor then computes the discount based on its internal logic and the type of customer or product it is dealing with.

This method decouples the discount calculation logic from the customer and product classes, making the system more flexible and easier to extend with new types of discounts or customer categories.

Real-Life Example

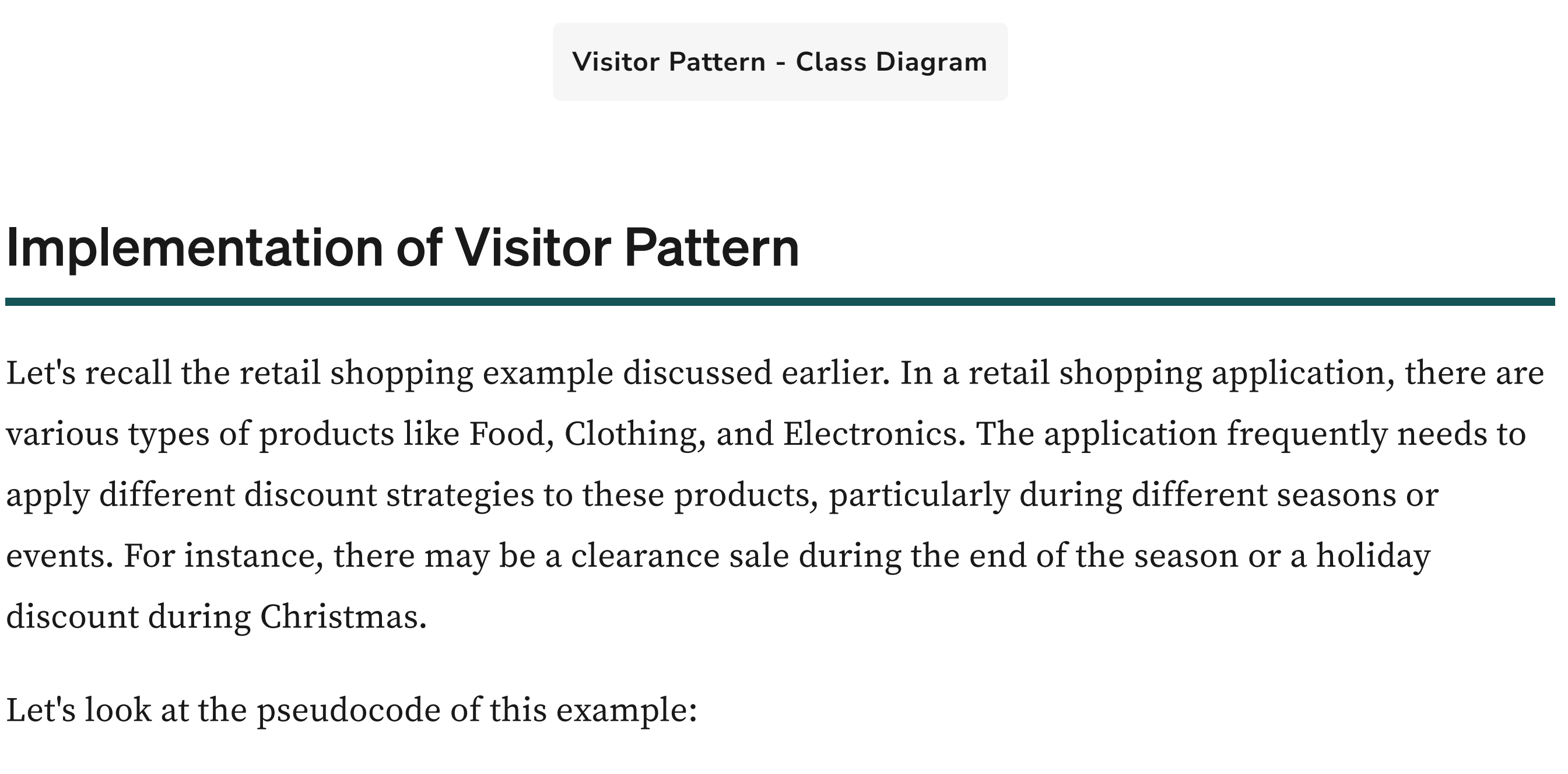
In software development, the Visitor pattern is often used in Integrated Development Environments (IDEs) for code analysis and refactoring. Consider an IDE that needs to handle various programming elements like classes, methods, and variables. Each element has its structure and properties.

Specialized visitors, like a **Code Analysis Tool** visitor, traverse these elements to perform actions like detecting code smells or suggesting optimizations. Another visitor, such as a **Refactoring Tool**, might visit the same elements to perform refactoring operations, like renaming variables or reorganizing classes.

Each visitor interacts with the elements differently based on the element's type and the visitor's purpose. For instance, the way a refactoring tool changes a class structure is different from how it alters a method. This setup allows the IDE to easily extend its functionality by adding new visitors for different tasks without altering the core structure of the programming elements, demonstrating the flexibility and scalability offered by the Visitor pattern.

Structure of Visitor Pattern

- **Visitor Interface**: Visitor interface or abstract class defines the interface to be implemented by concrete visitors, with each method corresponding to a specific element type.
- **Concrete Visitor**: The Concrete Visitor, another key participant, implements the Visitor interface, specifying operations to be performed on the elements. Multiple concrete visitors can exist, each providing a distinct operation.
- **Element Interface**: The Element Interface declares the accept method, enabling a visitor to visit the element by taking the visitor as an argument and invoking the appropriate method.
- **Concrete Element**: The Concrete Element implements the Element interface, defines the accept method, and calls the corresponding visit method on the visitor to perform operations on the element.
- **Object**: Finally, the Object Structure represents a collection or structure of elements accepting visitors and typically facilitates iteration over its elements.


Client
ObjectStructure
<<interface>>
Visitor
+ visit (ConcreteElement: object)
<< realize >>
Element
+ accept(visitor : object)
ConcreteVisitor
+ visit (ConcreteElement: object)
ConcreteElement
+ accept(visitor : object)

Visitor Pattern - Class Diagram

Implementation of Visitor Pattern

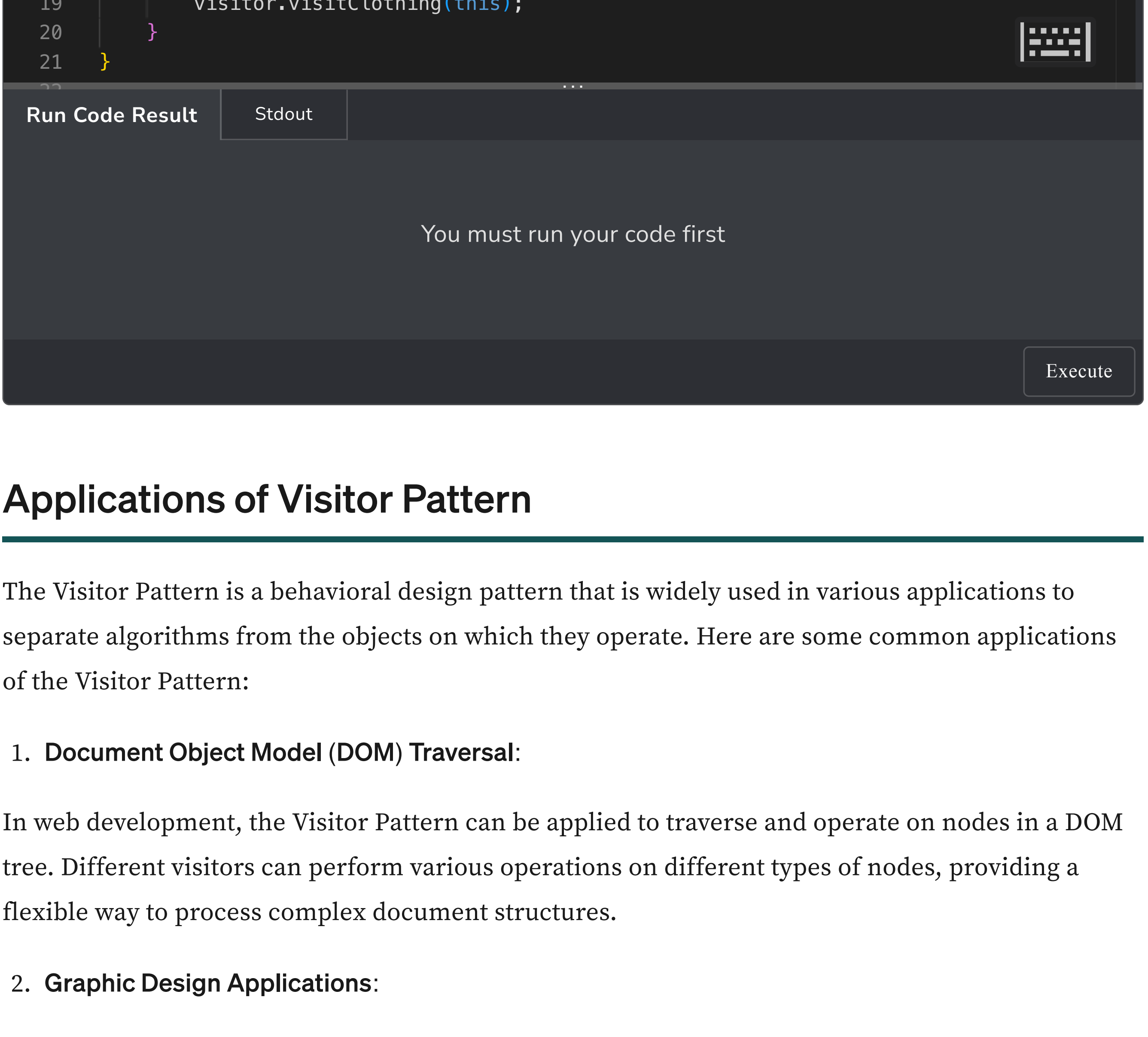
Let's recall the retail shopping example discussed earlier. In a retail shopping application, there are various types of products like Food, Clothing, and Electronics. The application frequently needs to apply different discount strategies to these products, particularly during different seasons or events. For instance, there may be a clearance sale during the end of the season or a holiday discount during Christmas.

Let's look at the pseudocode of this example:

```
// Visitor Interface
interface DiscountVisitor {
    visitFood(Food food)
    visitClothing(Clothing clothing)
    visitElectronics(Electronics electronics)
}

// Concrete Visitors
class HolidayDiscountVisitor implements DiscountVisitor {
    visitFood(food) {
        // Apply holiday discount logic for food
    }
    visitClothing(clothing) {
        // Apply holiday discount logic for clothing
    }
    visitElectronics(electronics) {
        // Apply holiday discount logic for electronics
    }
}

class ClearanceDiscountVisitor implements DiscountVisitor {
    visitFood(food) {
        // Apply clearance discount logic for food
    }
    visitClothing(clothing) {
        // Apply clearance discount logic for clothing
    }
    visitElectronics(electronics) {
        // Apply clearance discount logic for electronics
    }
}

// Element Interface
interface Product {
    accept(DiscountVisitor visitor)
}

// Concrete Elements
class Food implements Product {
    accept(visitor) {
        visitor.visitFood(this)
    }
}

class Clothing implements Product {
    accept(visitor) {
        visitor.visitClothing(this)
    }
}

class Electronics implements Product {
    accept(visitor) {
        visitor.visitElectronics(this)
    }
}

// Usage
food = new Food()
clothing = new Clothing()
electronics = new Electronics()

holidayVisitor = new HolidayDiscountVisitor()
clearanceVisitor = new ClearanceDiscountVisitor()

// Applying different discounts
food.accept(holidayVisitor)
clothing.accept(clearanceVisitor)
electronics.accept(holidayVisitor)
```

1. **Visitor Interface** (`DiscountVisitor`): This defines a contract for the visitors, listing methods for each type of visitable element (in this case, different product types).
2. **Concrete Visitors** (`HolidayDiscountVisitor`, `ClearanceDiscountVisitor`): Implementations of the visitor interface, each encapsulating the logic for a specific type of discount (e.g., holiday, clearance). Each visitor knows how to apply its discount to different product types.
3. **Element Interface** (`Product`): Defines an interface for elements that can be visited. The key method here is `accept`, which takes a visitor as an argument.
4. **Concrete Elements** (`Food`, `Clothing`, `Electronics`): These are the actual objects to which we want to apply operations (discounts, in this case). Each element implements the `accept` method to allow a visitor to perform an operation on it.
5. **Usage**: To apply a discount, you create an instance of a visitor (e.g., `HolidayDiscountVisitor`) and pass it to the `accept` method of a product. The visitor then applies the appropriate logic based on the type of product.

Implementation



```java
interface DiscountVisitor {
    void visitFood(Food food);
    void visitClothing(Clothing clothing);
    void visitElectronics(Electronics electronics);
}

interface Product {
    void accept(DiscountVisitor visitor);
}

class Food implements Product {
    public void accept(DiscountVisitor visitor) {
        visitor.visitFood(this);
    }
}

class Clothing implements Product {
    public void accept(DiscountVisitor visitor) {
        visitor.visitClothing(this);
    }
}
```

```
You must run your code first
```

Applications of Visitor Pattern

The Visitor Pattern is a behavioral design pattern that is widely used in various applications to separate algorithms from the objects on which they operate. Here are some common applications of the Visitor Pattern:

1. **Document Object Model (DOM) Traversal**:

In web development, the Visitor Pattern can be applied to traverse and operate on nodes in a DOM tree. Different visitors can perform various operations on different types of nodes, providing a flexible way to process complex document structures.

2. **Graphic Design Applications**:

In graphic design software, the Visitor Pattern can be employed to apply different filters or effects to various graphic elements. Visitors can encapsulate different image processing algorithms without modifying the classes of the graphic elements.

3. **Database Query Optimization**:

Visitors can be utilized in database query optimization. A visitor may traverse a query execution plan and apply optimization techniques to different types of query nodes, such as join operations or filtering conditions.

4. **Game Development**:

   In game development, the Visitor Pattern can be applied to traverse and operate on the elements of a game world. Different visitors can handle tasks such as rendering, collision detection, or artificial intelligence for different types of game objects.

5. **GUI Components**:

When working with graphical user interfaces, the Visitor Pattern can be used to perform operations on different types of GUI components. For instance, visitors can be employed to validate input fields, update the display of components, or handle user interactions.

6. **Abstract Syntax Tree (AST) Transformations**:

   In addition to compiler design, the Visitor Pattern is commonly employed in language processing to perform transformations on abstract syntax trees. Each visitor encapsulates a specific transformation, promoting modularity and extensibility.

These applications highlight the versatility of the Visitor Pattern in scenarios where there is a need to perform different operations on a set of related but distinct objects without modifying their class definitions

Pros and Cons

Pros	Cons
Separation of concerns: The Visitor Pattern separates the algorithms from the objects on which they operate. This promotes a clear separation of concerns and allows for the addition of new operations without modifying the object structure.	**Increased Complexity**: The Visitor Pattern can introduce complexity, especially in scenarios with a large number of different visitors and elements.
Open/Closed Principle: The pattern adheres to the Open/Closed Principle, which states that a class should be open for extension but closed for modification.	**Difficult to Modify**: Adding a new element to the object structure requires modifying the element interface and all existing visitors.
Increased Extensibility: The Visitor Pattern is highly extensible. It allows the addition of new visitors to perform different operations on the elements without modifying the existing elements or the visitor interface.	**Tight Coupling**: There is a potential for tight coupling between visitors and elements, as each visitor needs to know about all possible types of elements.
Increased Flexibility: The pattern provides flexibility by allowing different visitors to perform different operations on the same set of elements.	**Limited Type Safety**: The pattern might lead to limited type safety, especially when using a language without strict type checking.
Single Responsibility Principle: The pattern adheres to the Single Responsibility Principle by encapsulating each operation in a separate visitor.	**Increased Codebase**: The Visitor Pattern can lead to an increased number of classes, especially when there are numerous visitors and elements.

In conclusion, while the Visitor Pattern offers advantages in terms of separation of concerns, extensibility, and adherence to design principles, it also introduces complexities and potential challenges, especially in large and evolving systems. Careful consideration should be given to whether the benefits outweigh the drawbacks in a specific context.

← Previous
Template Method Pattern
Next →
Summary
☐ Mark as Completed